Pipeline 2.0

How the pipeline of tomorrow enforces security

Tomasz Świerszcz

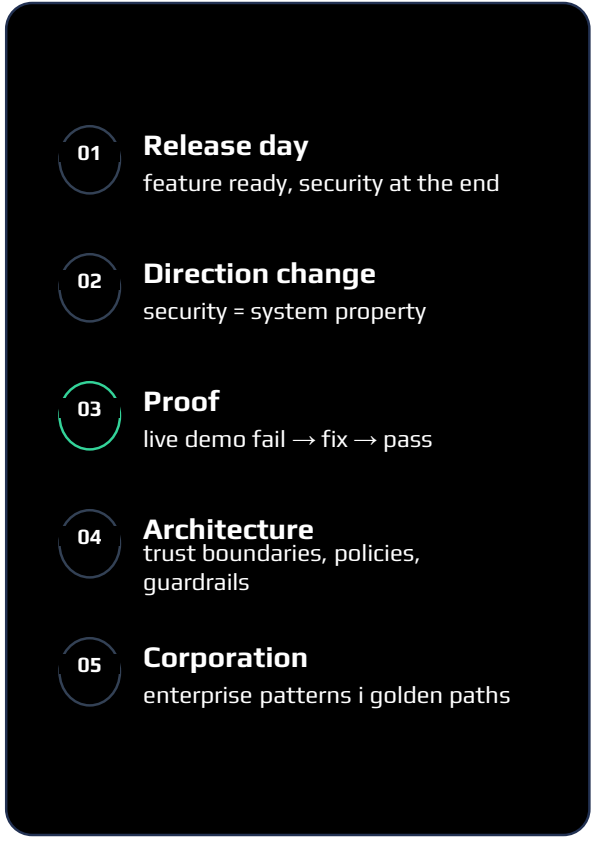


https://www.linkedin.com/in/tswierszcz/

tswierszcz@lafinstitute.org

Security is Architecture

The manufacturing process of tomorrow: SDLC, guardrails and pipeline,
that enforces the architecture — not just builds artifacts.



18:42. Release day.

The feature is ready. The tests are green. Business is waiting.
And then the classic one appears: "security has not yet worked".

- Costly correction instead of quick feedback
- Frustration: Dev vs Security
- Release becomes a negotiation, not a process



The real problem is not "do we have security?"
It is: "where in the system do we enforce the rules?"

"Security is not an SDLC stage. Security is a property of architecture."

If the security principle exists only in the document or in the head of an expert, it is the system that does not enforce it.

Document → recommendation · Pipeline → execution

From diagram to enforceable architecture

Three things an architect designs in the SDLC of tomorrow

Trust boundaries

Where does trust end and risk begin? Each transition between components is a candidate for control.

01

Threat model

STRIDE provides a language to talk about threats at the design stage, before vulnerable code is created.

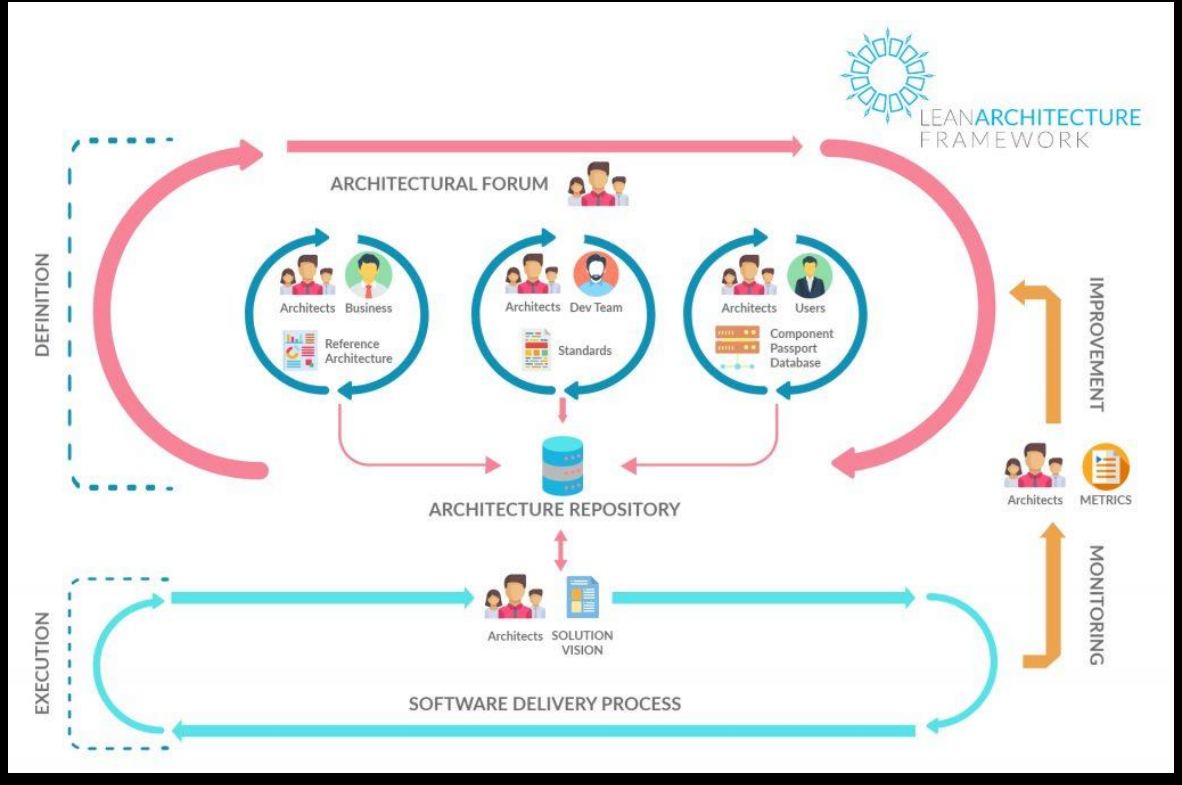
02

Security controls

AuthN, AuthZ, input validation, encryption, secrets, auditing — but as controls in the system, not just recommendations.

03

Architecture Governance Proces



Pipeline = enforcement architecture layer

The new SDLC does not ask for permission. It checks whether the system meets the rules.



Guardrails > gates

Gate is a man blocking a man. Guardrail is a system that guides the developer without discussing exceptions.

Demo · Step 1 - Vulnerable code

app/UserController.java

```java
public class UserController {
    public ResultSet getUser(Connection connection,
String userId) throws Exception {
        String query = "SELECT * FROM users WHERE id = "
+ userId;
        Statement stmt = connection.createStatement();
        return stmt.executeQuery(query);
    }
}
```

- "this is an ordinary, banal SQL concatenation"
- "The feature works — but the code is dangerous"
- "And that's why we want feedback before we get to production"

Demo · Step 2 — pipeline run

One command. This is what a process that can be trusted is supposed to look like.

terminal

```
$ cd demo-secure-sdlc
$ ./pipeline.sh

===> [1/3] Build
Build OK
===> [2/3] SAST - Semgrep
```

- There is no separate "Security Review Call"
- Control is part of the regular pipeline
- Feedback returns immediately to the author of the changes

Demo · Step 3 - Semgrep stops release

This is the moment when the pipeline says "no".

terminal · expected output

```
===> [2/3] SAST - Semgrep

app/UserController.java
   ❯❯❯ java.sql-injection
         Possible SQL injection via string
concatenation
         3 ┆ String query = "SELECT * FROM users
WHERE id=" + userId;

✕ 1 finding (pipeline stopped)
```

"We don't have a security team here. We have a pipeline that enforces the rules."

Demo · Step 4 - Fix without heroics

We don't code for long. Guardrail leads to a safe path.

fix code

terminal

```
$ cp
app_fixed/UserController.java
app/UserController.java
$ git diff --
app/UserController.java
```

app_fixed/UserController.java

```
1 public class UserController {
   public ResultSet getUser(Connection connection,
String userId) throws Exception {
       String query = "SELECT * FROM users WHERE id =
?";
       PreparedStatement stmt =
connection.prepareStatement(query);
       stmt.setString(1, userId);
       return stmt.executeQuery();
}
```

Why it's important

- The developer gets a clear direction
- The fix is local and fast
- The process promotes the right pattern

This is the guardrail: It doesn't just block — it leads.

Demo · Step 5 - Pipeline Passes

Same process. A different result. Because the artifact stopped breaking the rule.

terminal · expected output

```
$ ./pipeline.sh

===> [1/3] Build
Build OK
===> [2/3] SAST - Semgrep
No findings.
===> [3/3] Policy check - Conftest
0 tests failed

===> Pipeline OK
```

This is not a happy ending.

This is the proof that the process has become predictable.

Less negotiation.
More rule enforcement.

Policy as Code – Conftest

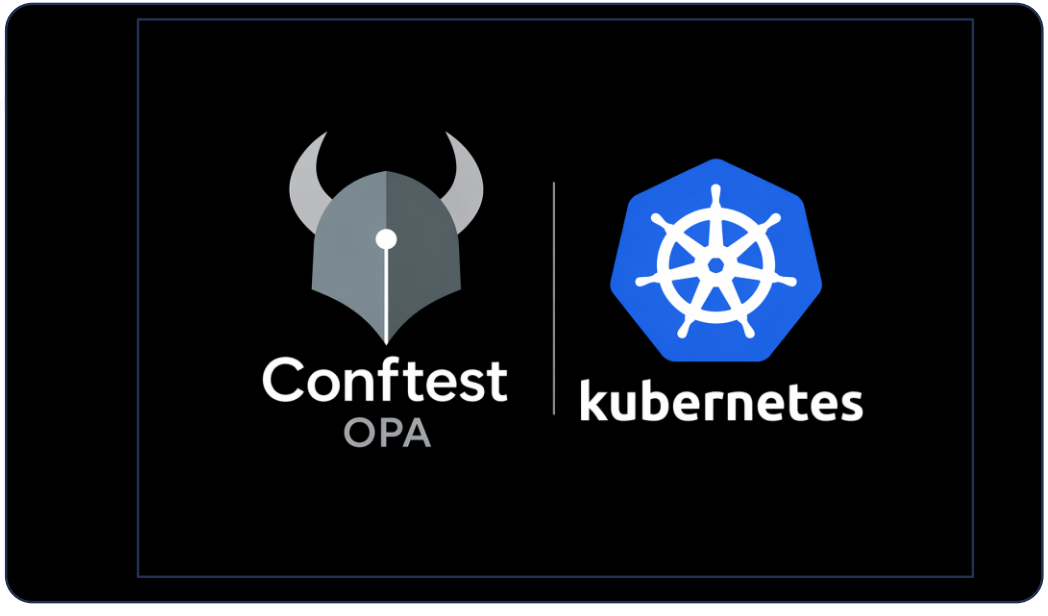
Pipeline enforces architecture – not man

Conftest in pipeline

✓ validates configuration (YAML / JSON)
✓ enforces architectural principles
✓ Blocks deployment for violations

01

02



Three architectural takeaways from this demo

This was not a demo of the tools. This was a demo of the system's responsibility.

Feedback has moved to the left

The problem went back to the author of the code in the same workflow.

01

The rule has become enforceable

Security has ceased to be an opinion. It became the result of an inspection.

02

Architecture has been given an execution mechanism

Pipeline confirms that the established rules are indeed in place.

03

How to grow from a single repo to an enterprise

This is not a "one-off demo". This is the operating pattern.

Mechanisms:

- threat model as an element of design
- repo templates and golden paths
- common rules Semgrep / secrets / IaC
- Policy Bundles for the Platform
- merics: fail rate, MTTR, exception debt

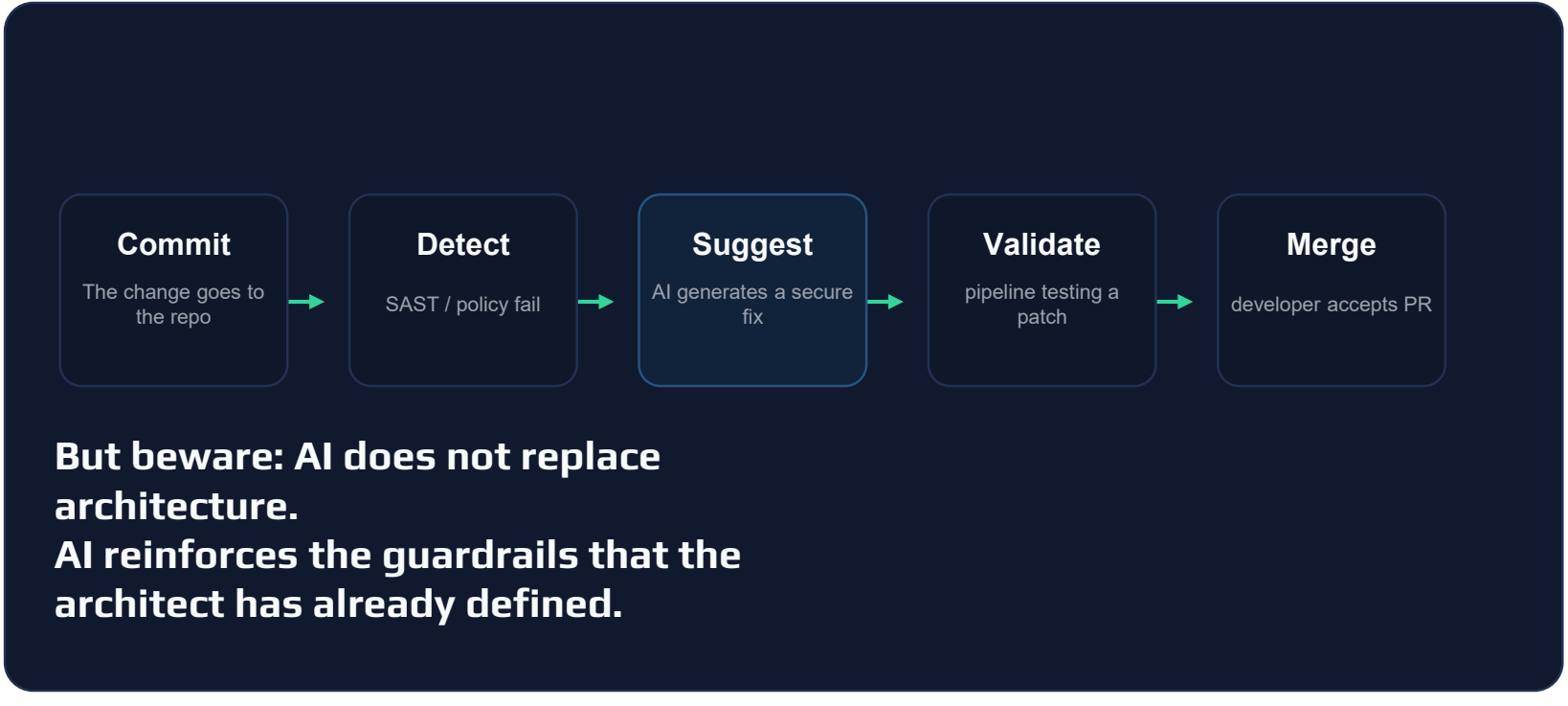
Changing the role of the architect

from the "diagram reviewer"
→ to "Guardrails"
→ to "Platform Constraints Designer"

The best architect of tomorrow does not draw the prettiest diagrams.
It designs rules that the system cannot break.

Next: self-healing pipelines i AI-assisted fixes

The next step is a process that not only detects — but also proposes a safe fix.



But beware: AI does not replace architecture.
AI reinforces the guardrails that the architect has already defined.

Security is not a human decision.

Security is the result of the system.

If the production process of tomorrow is to be faster, safer and less conflicting,
The architecture must be enforceable — and the pipeline must be able to say: pass or fail.

Dziękuję

Q&A

Questions and answers